Entregable Segundo Corte

SIS201-L – Programación Orientada a Objetos (laboratorio)
Universidad del Cauca

Modalidad	Individual
Entrega	Miércoles 11 de noviembre de 2026, 11:00 p. m.
Qué se entrega	El enlace a tu repositorio de GitHub, con la hoja de casos dentro
Calificación	De 1 a 100 puntos

1. Temas que se evalúan

1. Clases, objetos y encapsulamiento.
2. Constructores y destructores.
3. Organización de un programa en archivos `.h` y `.cpp`.
4. Herencia simple y constructores en la herencia.
5. Redefinición de métodos y funciones virtuales.
6. Clases abstractas y funciones virtuales puras.
7. Destructores virtuales.
8. Punteros a objetos y colecciones de punteros.
9. Memoria dinámica: `new` y `delete`.
10. Asociación y composición entre clases.
11. Excepciones propias: lanzar, propagar y capturar.
12. Sobrecarga del operador `«`.
13. Control de versiones con git.

2. Contexto

La caseta de peaje Norte de la Concesión Vial Cerro Verde necesita un programa de consola para llevar el registro de los vehículos que pasan y el dinero que se recauda. La concesión, sus nombres y todos los valores de este documento son inventados.

Esto es lo que pide la concesión:

R1. Por la caseta pasan carros, motos, buses y camiones. Todo vehículo que llega es de una de esas cuatro categorías. Cada uno se identifica por su placa, que nunca puede estar vacía.

R2. Además de la placa, cada categoría tiene sus propios datos. El carro lleva pasajeros (al menos uno) y puede subir otro pasajero. La moto tiene cilindraje (mayor que cero) y debe poder informar si es de alto cilindraje, es decir, de más de 400. El bus tiene puestos (al menos uno). El camión tiene ejes (al menos dos) y se le puede agregar un eje.

R3. Cada categoría paga un valor distinto por pasar, según la tabla de la sección siguiente. El camión paga por cada eje.

R4. Un vehículo puede tener un propietario o no tenerlo, y el propietario se puede cambiar. Algunos propietarios están inscritos como vecinos de la concesión. Los carros y las motos de un vecino pagan con descuento. Los buses y camiones no.

R5. Las personas que aparecen en el sistema son los propietarios y los operadores. Todas tienen nombre y documento, y cualquiera puede presentar una descripción de sí misma: el propietario indica si es vecino y el operador muestra su código de empleado.

R6. La caseta la atiende un operador. Cada vez que pasa un vehículo, la caseta guarda un registro del paso con la placa, la categoría y el valor cobrado en ese momento. La caseta sabe cuántos pasos lleva, cuánto dinero ha recaudado y cuántos vehículos de cada categoría han pasado.

R7. Al terminar la jornada, el operador cuenta el efectivo que tiene en caja y lo declara a la caseta. Si coincide con lo recaudado, la caja cuadra. Si no coincide, el sistema avisa cuál es la diferencia.

R8. Un vehículo se puede mostrar en pantalla con un solo `cout`, con su categoría, su placa y sus datos propios.

R9. Cuando un dato es inválido, el sistema no sigue adelante con ese dato y avisa a quien lo llamó. Los mensajes que ve el usuario los muestra el programa principal, no las clases.

3. Tarifas de tu variante

Toma los tres últimos dígitos de tu código estudiantil. En este documento se llaman c (centena), d (decena) y u (unidad). Escribe tu código en la primera línea de tu `LEEME.md`.

Concepto	Valor	Ejemplo con 427
Carro	$10000 + 200\,u$	11400
Moto	$3000 + 100\,u$	3700
Bus	$14000 + 300\,d$	14600
Camión, por cada eje	$8000 + 100\,d$	8200
Descuento de vecino (carros y motos)	$(30 + c)\,\%$	34 %

El descuento se calcula con números enteros: `tarifa * (100 - descuento) / 100`. Con 427, un carro de un vecino paga $11400 \cdot 66/100 = 7524$.

4. Condiciones obligatorias

C1. La clase base de los vehículos se llama `Vehiculo`. No debe poder crearse un objeto de esa clase directamente. El valor que paga cada vehículo lo calcula su propia categoría y se pide de la misma forma sin importar la categoría. El cobro no recibe parámetros.

C2. La clase base de las personas se llama `Persona`. La descripción de una persona se pide de la misma forma sin importar si es propietario u operador.

C3. Los vehículos y las personas se crean con `new` en el programa principal y se guardan en colecciones de punteros (`vector<Vehiculo *>` y `vector<Persona *>`). Al terminar, se liberan con `delete` usando el puntero de la clase base.

C4. La caseta crea cada registro de paso con `new` y lo libera en su destructor. La caseta no libera los vehículos ni las personas.

C5. Los destructores de todas las clases de vehículos, de `Caseta` y de la clase del registro de paso imprimen una línea con la forma `Destruyendo Carro ABC123`. Es lo único que las clases pueden imprimir.

C6. Las excepciones son clases propias que heredan de `std::exception` y se capturan por referencia constante. Son tres: placa inválida, datos inválidos y descuadre de caja. Van en `Excepciones.h` y `Excepciones.cpp`.

C7. Cada clase va en su propio par de archivos `.h` y `.cpp`. Los `.h` van en la carpeta `include/` y los `.cpp` en `src/`.

C8. El programa principal ejecuta los casos de la hoja y muestra una línea por cada uno que empiece con `Caso n:`, donde n es el número del caso. Puede tener además un menú o cualquier otra forma de uso.

Para compilar, desde la carpeta del proyecto:

```
g++ -std=c++17 -Iinclude src/*.cpp -o build/peaje
```

En Mac usa `clang++`. En PowerShell, si el comodín no funciona, escribe los `.cpp` uno por uno.

5. Repositorio

Crea tu propio repositorio en GitHub, desde la consola o desde GitHub Desktop, y ve guardando el avance con commits mientras trabajas. Haz al menos un commit al terminar cada grupo de clases (excepciones, personas, vehículos, caseta), otro con la hoja de casos y otro con el `LEEME.md`. El repositorio puede ser privado: agrega al profesor como colaborador, cuyo usuario se dice en clase.

Estructura:

```
peaje/
  include/ los .h
  src/ los .cpp y main.cpp
  hoja/ fotografia o escaneo de la hoja de casos
  build/ ejecutables (va en .gitignore)
  LEEME.md
```

El `LEEME.md` tiene, en este orden:

1. Tu código estudiantil, en la primera línea.
2. Por qué no se puede crear un objeto de `Vehiculo` directamente.
3. Por qué el destructor de `Vehiculo` es virtual.
4. Qué hace tu programa si se llama al registro de un paso con un puntero nulo, y por qué decidiste eso.

Cada punto en máximo tres líneas.

6. Hoja de casos

En el Anexo hay una hoja con 13 casos. Imprímela o cópiala a mano. Antes de ejecutar tu programa, calcula a mano el resultado esperado de cada caso con la tabla de tu variante y escríbelo en la columna correspondiente. Después ejecuta tu programa y escribe en la otra columna lo que imprimió. Si no coinciden, corrige tu programa o tu cálculo, pero deja visible lo que escribiste primero. Entrega una fotografía o un escaneo legible dentro de la carpeta `hoja/`.

7. Uso de inteligencia artificial

Está prohibido usar herramientas de inteligencia artificial en cualquier parte de este entregable. Esto incluye chatbots, asistentes dentro del editor, autocompletado con inteligencia artificial y agentes de código. En particular, no se permite:

- Generar código, completo o por partes: clases, métodos, el programa principal, los `.h` o el `LEEME.md`.
- Pedir o generar el diseño: qué clases crear, cómo se relacionan, en qué orden se hacen las llamadas, el flujo del programa, diagramas o pseudocódigo.
- Pegar tu código o los errores del compilador para que los corrijan o los expliquen.
- Reescribir, traducir o "mejorar" código que ya escribiste.
- Calcular, completar o revisar la hoja de casos.

Sí puedes usar tus apuntes, el material y los ejemplos vistos en clase, y la documentación del lenguaje.

Se revisan el código, el historial del repositorio, la hoja y el `LEEME.md`. Si el entregable muestra alertas de uso de inteligencia artificial, puede ser anulado. Son alertas, entre otras: código con un estilo o con construcciones que no se han visto en el curso, bloques grandes de código que aparecen de golpe en un solo commit, o una hoja que no corresponde con lo que hace el programa. Copiar el trabajo de otro estudiante o dejar que otro copie el tuyo también causa la anulación.

8. Rúbrica

Criterio	Puntos
Jerarquía de vehículos: clase abstracta, constructores que llaman a la base, redefiniciones y funciones virtuales	15
Personas y asociación entre vehículo y propietario: herencia, descripción virtual, descuento de vecino	10
Caseta y registro de paso: `new`, `delete`, destructores y composición	15
Excepciones: clases propias, lanzamiento, captura en el programa principal y mensajes	10
Sobrecarga de `«`, archivos `.h` y `.cpp` en `include/` y `src/`, y compilación sin errores	10
El programa principal ejecuta y muestra los 13 casos	10
Hoja de casos hecha a mano, calculada antes de ejecutar y coherente con el programa	15
Historial de commits repartido y con sentido	5
`LEEME.md`	5
Claridad del código: nombres, sin código repetido, formato consistente	5
Total	100

Anexo: hoja de casos

Código estudiantil: _____ Últimos tres dígitos: _____

Caso	Situación	Resultado esperado (a mano)	Lo que imprime mi programa
1	Carro ABC123, 3 pasajeros, propietario no vecino. ¿Cuánto paga?		
2	El mismo carro, ahora con un propietario vecino. ¿Cuánto paga?		
3	Moto XYZ45A de 125 cc, propietario vecino. ¿Cuánto paga?		
4	Moto MOT650 de 650 cc, sin propietario. ¿Cuánto paga? ¿Es de alto cilindraje?		
5	Bus BUS777 de 38 puestos, propietario vecino. ¿Cuánto paga?		
6	Camión CAM301 de 3 ejes, sin propietario. ¿Cuánto paga?		
7	El mismo camión después de agregarle un eje. ¿Cuánto paga?		
8	Crear un carro con la placa vacía. ¿Qué excepción se lanza?		
9	Crear un camión de 1 eje. ¿Qué excepción se lanza?		
10	Pasan por la caseta los vehículos de los casos 2, 3, 5 y 7. ¿Cuál es el recaudo total y cuántos hay de cada categoría?		
11	El operador declara exactamente el recaudo del caso 10. ¿Qué ocurre?		
12	El operador declara 1000 menos que el recaudo del caso 10. ¿Qué ocurre?		
13	Al terminar el programa se libera la caseta y luego los vehículos del caso 10 a través de `Vehiculo *`. ¿En qué orden aparecen los mensajes de destrucción?		